Shiny apps

To build a shiny app we must first understand how it works. A shiny app consists of two core components: a server and a user interface (UI). In this section, we will explain how they are connected and work together to create the shiny app.

Learning objectives

The learning objectives for this session are:

1. Objective one

The server and the UI

A shiny app can take a static dataset and make it available for the user to explore. Depending on how you build the app, you can allow the user to search for specific search terms, compare expression levels, or explore interactive figures to find new genes affected in your dataset.

:speech_balloon: Discussion activity: Title

Time: \3 minutes.

Discuss with the person next to you - what kind of data do you have that could benefit from being interactive? Do you have an idea for an app you want to make?

Reading task: ~4 minutes

A shiny app can get incredibly complex and consist of many interconnected scripts. But at it's simplest, a shiny app consists of two elements that work together.

At it's most simple, a shiny app can be one simple function. The following code will create a shiny app (although an empty one):

```
app <- shiny::shinyApp(
  ui = shiny::fluidPage(),
  server = function(input, output){}
)
shiny::runApp(app)
```

The UI (short for user interface) is what we see on the screen. The code contains information about what is shown on the page (such as plots, buttons and images) and in what layout (organized in columns, with a navigation bar, etc.).

The server contains information about the objects in the UI, and how the app should respond to user input. It contains information about what data is displayed in the plot, what happens when you press a button, and how the app should update when specific events happen.

A major part of building a shiny app is understanding the interplay between the UI and the server.

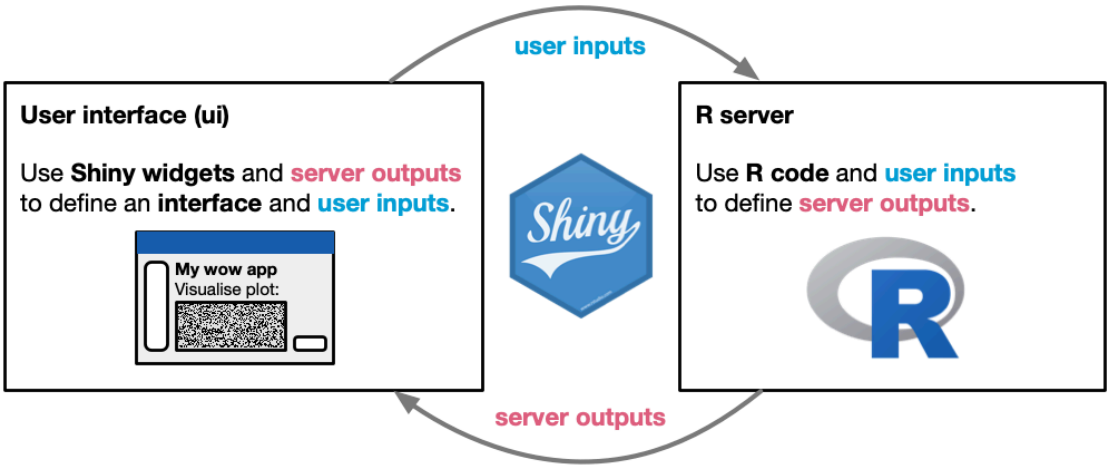


Figure 1: A graphical representation of the interplay between the UI and the server. From introduction to data science

:book: Adding elements to our UI

Time: \6 minutes.

Shiny comes with a lot of options for building a user interface. Often you can add plots directly into a Shiny interface. lets say we want to make a graph from the classical `iris` data set and add it into our empty app above.

```
iris_plot <- ggplot2::ggplot(iris,
                             ggplot2::aes(x = Sepal.Length, y = Petal.Length, color =
Species)) +
  ggplot2::geom_point()

iris_plot
```

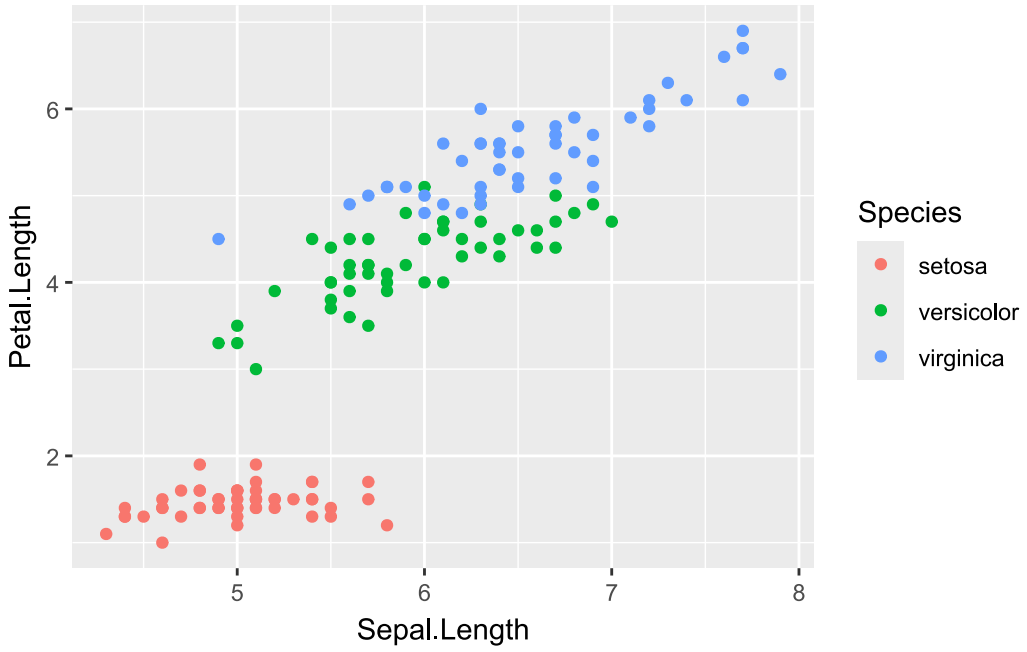


Figure 2: Our classical iris plot

To add this to our app, we need to tell Shiny two things: that we want a page in our page layout, and what we want the plot to contain.

```
app <- shiny::shinyApp(
  ui = shiny::fluidPage(
    shiny::plotOutput(outputId = "iris_plot"
    )
  ),
  server = function(input, output){
    output$iris_plot <- shiny::renderPlot(
      iris_plot
    )
  }
)
shiny::runApp(app)
```

If we take the code above and paste it into the console, we should get an app with the plot. We have now created content for our app!

BSlib

It is entirely possible to build a shiny app out of components from the shiny package itself. However, the "modern" shiny design focuses on a different package called "bslib" which provides access to the highly flexible bootstrap framework. Bslib allows a number of UI components with a high degree of customization, and which renders nicely, even on a small screen such as a phone.

Bslib features both pages (used to define the overall rules for our app page) and the components of the pages (especially as units of UI elements called "cards"). We can take our previous code and modify it to become a navbar page.

```
app <- shiny::shinyApp(
  ui = bslib::page_sidebar(
    title = "Our Iris plot",
    sidebar = "here is our sidebar",
    shiny::plotOutput(outputId = "iris_plot"
    )
  ),
  server = function(input, output){
    output$iris_plot <- shiny::renderPlot(
      iris_plot
    )
  }
)
shiny::runApp(app)
```

If we paste this code into our console and runs it (rerunning the code for the iris plot if necessary), we will see we now have a sidebar where we can put something in. Let's leave it empty for now, and instead focus on the other functionality of bslib we teased before - the cards.

Cards are an effective way to organize plots and other UI components. They are easy to implement - we simply wrap our UI content (in this case the Iris plot) in `bslib::card()`

```
app <- shiny::shinyApp(
  ui = bslib::page_sidebar(
```

```
    title = "Our Iris plot",
    sidebar = "here is our sidebar",
    bslib::c(shiny::plotOutput(outputId = "iris_plot")
    )
  ),
  server = function(input, output){
    output$iris_plot <- shiny::renderPlot(
      iris_plot
    )
  }
)
shiny::runApp(app)
```

In itself this does not do much. However, cards have some attributes such as headers that can be modified if we for example want to give our plot a header.

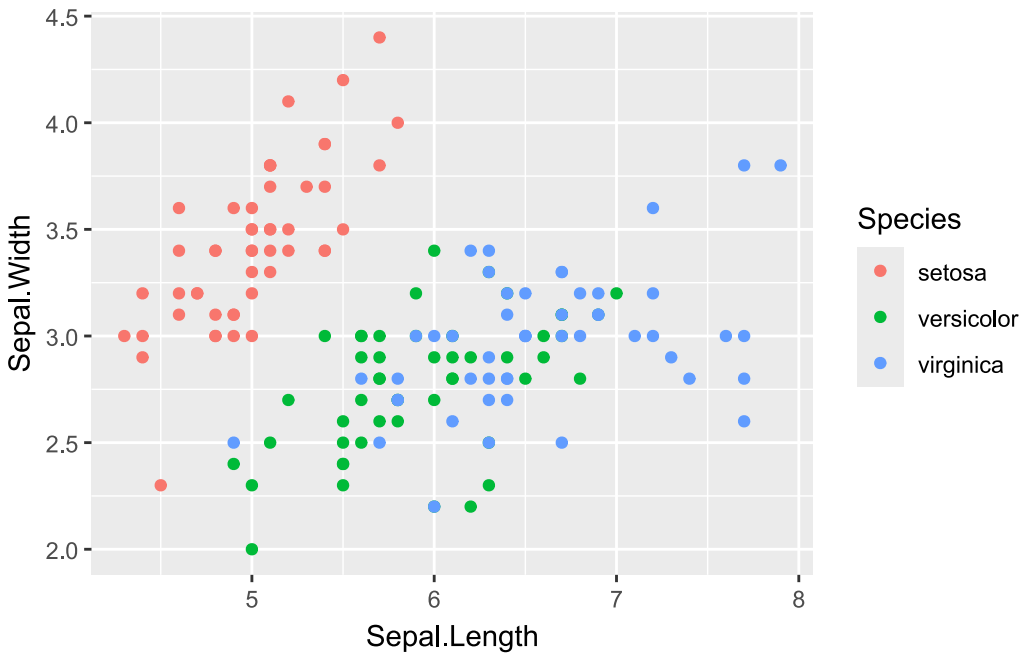
```
app <- shiny::shinyApp(
  ui = bslib::page_sidebar(
    title = "Our Iris plot",
    sidebar = "here is our sidebar",
    bslib::card(
      bslib::card_header("A beautiful plot about flowers"),
      shiny::plotOutput(outputId = "iris_plot")
    )
  ),
  server = function(input, output){
    output$iris_plot <- shiny::renderPlot(
      iris_plot
    )
  }
)
shiny::runApp(app)
```

The true power of cards becomes evident once we start having several of them. Let's make a few more plots so we can see how bslib behaves once we have multiple:

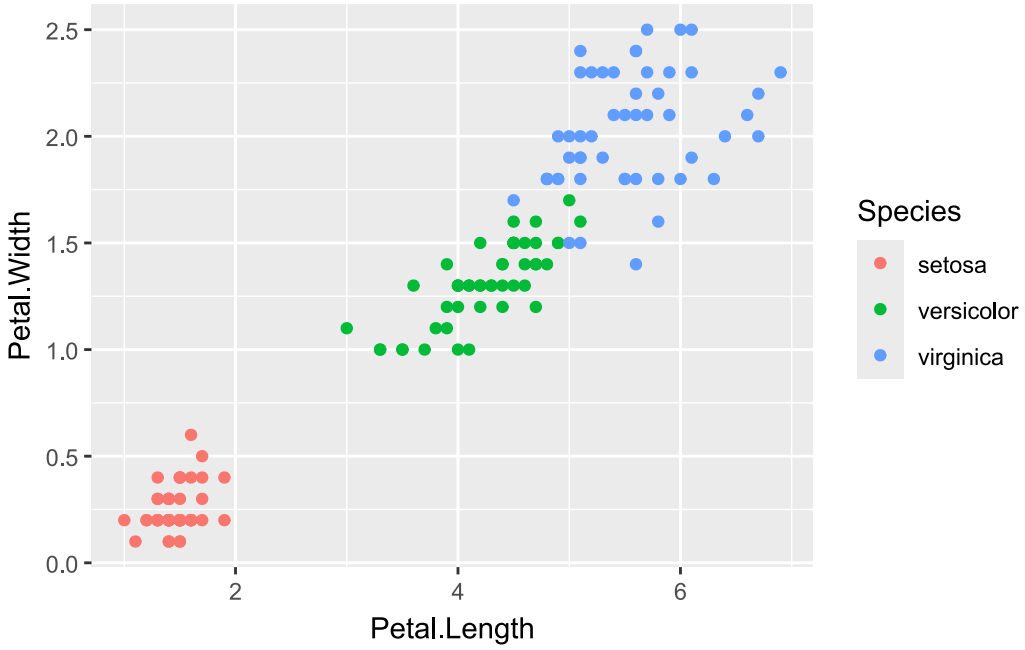
```
iris_plot_2 <- ggplot2::ggplot(iris,
                               ggplot2::aes(x = Sepal.Length, y = Sepal.Width, color =
Species)) +
  ggplot2::geom_point()


iris_plot_3 <- ggplot2::ggplot(iris,
                               ggplot2::aes(x = Petal.Length, y = Petal.Width, color =
Species)) +
  ggplot2::geom_point()

iris_plot_2
```




```
iris_plot_3
```



Lets add our two new plots to our layout, copy it to the console, and run it:

```
app <- shiny::shinyApp(
  ui = bslib::page_sidebar(
    title = "Our Iris plot",
    sidebar = "here is our sidebar",
    bslib::card(
      bslib::card_header("A beautiful plot about flowers"),
      shiny::plotOutput(outputId = "iris_plot")
    ),
    bslib::card(
```

```
      bslib::card_header("Another beautiful plot about flowers"),
      shiny::plotOutput(outputId = "iris_plot_2")
    ),
    bslib::card(
      bslib::card_header("OKay I think I've had enough flower plots for now"),
      shiny::plotOutput(outputId = "iris_plot_3")
    )
  ),
  server = function(input, output){
    output$iris_plot <- shiny::renderPlot(
      iris_plot
    )
    output$iris_plot_2 <- shiny::renderPlot(
      iris_plot_2
    )
    output$iris_plot_3 <- shiny::renderPlot(
      iris_plot_3
    )
  }
)
shiny::runApp(app)
```

It looks nice! if we want to change the order of our cards, we can add another layer from bslib, and define how many columns we want. We wrap all our cards in a function called `bslib::layout_column_wrap()` and add the argument `width = 1/2` to get 2 columns (if this is indeed what we want). If we instead had written `width = 1/3` we would get 3 columns and so forth.

```
app <- shiny::shinyApp(
  ui = bslib::page_sidebar(
    title = "Our Iris plot",
    sidebar = "here is our sidebar",
    bslib::layout_column_wrap(
      width = 1/2,
      bslib::card(
        bslib::card_header("A beautiful plot about flowers"),
        shiny::plotOutput(outputId = "iris_plot")
      ),
      bslib::card(
        bslib::card_header("A beautiful plot about flowers"),
        shiny::plotOutput(outputId = "iris_plot_2")
      ),
      bslib::card(
        bslib::card_header("A beautiful plot about flowers"),
        shiny::plotOutput(outputId = "iris_plot_3")
      )
    )
  ),
  server = function(input, output) {
    output$iris_plot <- shiny::renderPlot(iris_plot)
    output$iris_plot_2 <- shiny::renderPlot(iris_plot_2)
    output$iris_plot_3 <- shiny::renderPlot(iris_plot_3)
```

```
  }
)
shiny::runApp(app)
```

There is much more functionality to building a good user interface and there are many resources on bslib. But for now we know how to build a Shiny interface

Summary

In this session we:

- Learned about UI and server
- Explored the relationship between outputs and render functions
- Introduced bslib and showed how it can be used to make pages and UI elements

Next session, we will show how we can build a shiny app so we have a place for our UI